Triplet Cross Memory Bank (Hardest Positive & Hardest Negative)

1) Current Mini-Batch (size = B)


x_1
[1, 0, 1, 0, 1, ...]


x_2
[0, 1, 0, 1, 0, ...]

⋮


x_B
[0, 0, 1, 0, 1, ...]

Encoder
$f(\cdot)$

Batch Embeddings

$E_{batch} \in \mathbb{R}^{B \times D}$

e_1 (D-dim)
e_2 (D-dim)
⋮
e_B (D-dim)

2) Cross Memory Bank (Global)

$M \in \mathbb{R}^{N \times D}$ (N >> B)

m_1 (label: [1, 0, 1, 0, ...])
m_2 (label: [1, 0, 1, 0, ...])
m_3 (label: [0, 1, 0, 1, ...])
⋮
m_N (label: [0, 0, 1, 0, ...])

Updated with Momentum
(enqueee / dequeue)

3) Distance Matrix

$dist \in \mathbb{R}^{B \times N}$

	m_1	m_2	m_3	...	m_N
e_1	0.21	0.13	0.84	...	0.65
e_2	0.72	0.61	0.18	...	0.90
⋮	⋮	⋮	⋮	...	⋮
e_B	0.33	0.71	0.26	...	0.77

Lower = More similar
(Cosine Distance)

4) For each anchor e_i (from batch)

Positive indices (same class)

pos_idx = [1, 2, 5, 8, ...]

dist[i][pos_idx]

0.21 | 0.13 | 0.19 | 0.28 | ...

argmin → index: 2

Negative indices (diff class)

neg_idx = [3, 4, 6, 7, ...]

dist[i][neg_idx]

0.84 | 0.73 | 0.91 | 0.55 | ...

argmax → index: 6

Hardest Positive
(min distance)
m_2

Hardest Negative
(max distance)
m_6

How we select:

```
pos_probs = dist[i][pos_idx]
neg_probs = dist[i][neg_idx]

hardest_pos = mem_emb[pos_idx[torch.argmin(pos_probs)]]
hardest_neg = mem_emb[neg_idx[torch.argmax(neg_probs)]]
```

Anchor
(from batch)
e_i

Hardest Positive
(closest same-class)
m_{pos} (= m_2)

Hardest Negative
(furthest diff-class)
m_{neg} (= m_6)

Small distance
$d(a, p)$

Large distance
$d(a, n)$

Overall Training Flow



5) Triplet Loss

$$L = \max(0,\ d(a, p) - d(a, n) + \text{margin})$$

$d(a, p)$: distance to hardest positive (small)
$d(a, n)$: distance to hardest negative (large)

Key Points

- ✓ Cross Memory Bank → large pool
- ✓ Hardest Positive = min distance same class
- ✓ Hardest Negative = max distance different class
- ✓ Makes triplet learning stronger